

```
{ "cells": [ { "cell_type": "markdown", "metadata": { "id": "X9V5ptJAIEbw", "source": [ "# FIT5197 2026 S1 Final Assessment", "", "SPECIAL NOTE: Please refer to the assessment page (Clayton) (https://learning.monash.edu/mod/assign/view.php?id=5333168), or (Malaysia) (https://learning.monash.edu/mod/assign/view.php?id=4851875&forceview=1) for rules, general guidelines and marking rubrics of the assessment (the marking rubric for the kaggle competition part will be released near the deadline - around 5 working days - in the same page). Failure to comply with the provided information will result in a deduction of mark (e.g., late penalties) or breach of academic integrity." ] }, { "attachments": { "image-3.png": { "image/png": "iVBORwOKGGoAAAANSUHEUgAAAFmCAIAAADSzEH0AAAgAEIEQVR4Aey9dVRUa9vA/f77fWt9632eY8DUztnTdlsgGBgYKHaLndiFamKhAqJioNjHTkQCQ", "image-4.png": { "image/png": "iVBORwOKGGoAAAANSUHEUgAAAYsAAAE7CAIAAACjSDqpAAAgAEIEQVR4Aey9B1QUS6L4ved83/977/92r0qY2D05kXNOiCSg2KOGDChqNeECTPmHDAOCi", "cell_type": "markdown", "metadata": { "id": "FkFAwCwb7is", "source": [ "PLEASE ENSURE YOU READ AND COMPLETE ALL REQUIRED INFORMATION BELOW BEFORE STARTING YOUR ASSIGNMENT, AS FAILURE TO DO SO WILL RESULT IN A MARK OF ZERO.", "", "YOUR NAME: SUN ZHEN", "", "STUDENT ID: 36446874", "", "KAGGLE NAME (required for marking - see Part 1, Question 5 or Part 2): xiamiya", "", "Important: Your Kaggle name must be identical (including capitalization, spacing, and special characters) across both Regression and Classification contests. Marks will be deducted if your Kaggle name is inconsistent or missing. No exceptions will be made!", "", "You can find your Kaggle name by clicking on your profile icon at the top-right corner of the Kaggle website and go to your profile. For example, here is a screenshot of where to find it:", "", "image-4.png", "", "After that, the Kaggle name can be found similar to the one in the picture:", "", "image-3.png", "", "Please note that the name to be recorded is not \"Dan Nguyen 11\" but \"dangnguyen11\". Marks will be deducted if you record the incorrect names.", "", "Please also enter your details in this google form (https://forms.gle/M5F3RERccoM7MuYE9).", " ] }, { "cell_type": "markdown", "metadata": { "id": "f_u6Dyj1xehk", "source": [ "# Part 1 Regression (50 Marks)", " ] }, { "cell_type": "markdown", "metadata": { "id": "marked-instrument", "source": [ "A few thousand people were questioned in a life and wellbeing survey to build a model to predict happiness of an individual. You need to build regression models to optimally predict the variable in the survey dataset called 'happiness' based on any, or all, of the other survey question responses.", "", "You have been provided with two datasets, regression_train.csv and regression_test.csv. Using these datasets, you hope to build a model that can predict happiness level using the other variables. regression_train.csv comes with the ground-truth target label (i.e. happiness level) whereas regression_test.csv comes with independent variables (input information) only.", "", "On the order of around 70 survey questions have been converted into predictor variables that can be used to predict happiness. We do not list all the predictor names here, but their names given in the data header can clearly be linked to the survey questions. e.g., the predictor variable 'iDontFeelParticularlyPleasedWithTheWayIam' corresponds to the survey question 'I don't feel particularly pleased with the way I am.'", " ] }, { "cell_type": "markdown", "metadata": { "id": "3-Gbtyt1g7iv", "source": [ "PLEASE NOTE THAT THE USE OF LIBRARIES ARE PROHIBITED IN THESE QUESTIONS UNLESS STATED OTHERWISE, ANSWERS USING LIBRARIES WILL RECEIVE 0 MARKS", " ] }, { "cell_type": "markdown", "metadata": { "id": "wound-marriage", "source": [ "## Question 1 (NO LIBRARIES ALLOWED) (4 Mark)", "Please load the regression_train.csv and fit a", "textbf{multiple linear regression model", " (https://en.wikipedia.org/wiki/Linear\_regression) with 'happiness' being the target variable. According to the summary table, which predictors do you think are possibly associated with the target variable (use the significance level of 0.01), and which are the Top 5 strongest predictors? Please write an R script to automatically fetch and print this information.", "", "NOTE: Manually doing the above tasks will result in 0 marks.", " ] }, { "cell_type": "code", "execution_count": 1, "metadata": { "vscode": { "languageId": "r", "outputs": [ { "name": "stdout", "output_type": "stream", "text": [ "Predictors possibly associated with happiness at alpha = 0.01:", " Predictor Estimate t value", " income80k - 120k 37.280636 22.457548", " income50k - 80k 28.702333 16.476562", " income200k above 20.030623 14.487211", " income20k - 50k 20.819954 13.114552", " income15k - 20k 15.407464 11.991560", " income120k - 150k 11.476893 9.279294", " income150k - 200k 11.457538 9.239101", " income10k - 15k 6.285853 5.058513", " alwaysStressed -1.341180 -3.882886", " whatIsYourHeightExpressItAsANumberInMetresM180 - 185 6.940227 2.915956", " Pr(>|t|)", " 1.419507e-74", " 1.115904e-47", " 4.777672e-39", " 2.606555e-33", " 8.481388e-29", " 8.297612e-19", " 1.135908e-18", " 6.259097e-07", " 1.193717e-04", " 3.730638e-03", "", "Top 5 strongest predictors by absolute t-statistic:", " Predictor Estimate t value Pr(>|t|)", " income80k - 120k 37.28064 22.45755 1.419507e-74", " income50k - 80k 28.70233 16.47656 1.115904e-47", " income200k above 20.03062 14.48721 4.777672e-39", " income20k - 50k 20.81995 13.11455 2.606555e-33", " income15k - 20k 15.40746 11.99156 8.481388e-29", "", "Full model R-squared: 0.7665", " Adjusted R-squared: 0.7303", " ] }, { "source": [ "# Q1: Multiple linear regression using base R only.", "train <- read.csv(\"regression_train.csv\")", "", "lm.fit <- lm(happiness ~ ., data = train)", "model.summary <- summary(lm.fit)", "coef.table <- as.data.frame(model.summary", "coefficients)\n", " ", "coef.table", "Predictor <- rownames(coef.table)", "", "# Remove the intercept because the question asks for associated predictors.", "coef.table <- coef.table[coef.table", "Predictor! = \"(Intercept)\", ]\n", " ", "significant.predictors <- significant.predictors[order(significant.predictors", "Pr(> |t|), )\n", " ", " ", "top5.strongest <- -coef.table[order(abs(coef.table", "t value", " decreasing = TRUE), [1:5, ], "", "cat(\"Predictors possibly associated with happiness at alpha = 0.01:\n\"),", "print(significant.predictors[, c(\"Predictor\", \"Estimate\", \"t value\", \"Pr(>|t|)\")],", "row.names = FALSE)", "", "cat(\"\\nTop 5 strongest predictors by absolute t-statistic:\n\"),", "print(top5.strongest[, c(\"Predictor\", \"Estimate\", \"t value\", \"Pr(>|t|)\")],", "row.names = FALSE)", "", "cat(\"\\nFull model R-squared:\", round(model.summary", "r.squared, 4), \n", "n)\n", " ", "cat(\"\\nAdjusted R - squared : \n\", round(model.summary", "adj.r.squared, 4), \"\\n\")", " ] }, { "cell_type": "markdown", "metadata": { "id": "royal-finland", "source": [ "## Question 2 (2 Mark)", "R squared (https://en.wikipedia.org/wiki/Coefficient\_of\_determination) from the summary table reflects that the full model doesn't fit the training dataset well; thus, you try to quantify the error between the values of the ground-truth and those of the model prediction. You want to write a function to predict 'happiness' with the given dataset and calculate the root mean squared error (RMSE) (https://en.wikipedia.org/wiki/Root-mean-square\_deviation) between the model predictions and the ground truths. Please test this function on the full model and the training dataset.", " ] }, { "cell_type": "code", "execution_count": 2, "metadata": { "vscode": { "languageId": "r", "outputs": [ { "name": "stdout", "output_type": "stream", "text": [ "Training RMSE for the full linear model: 6.7017", " ] }, { "source": [ "# Q2: Prediction and RMSE function.", "rmse <- function(predicted, actual) {", "sqrt(mean((predicted - actual)^2)),", " },", "predict_and_rmse <- function(model, data, target_name = \"happiness\") {", "predicted <- predict(model, newdata = data)", "actual <- data[[target_name]]", "rmse(predicted, actual)", " },", "full.model.rmse <- predict_and_rmse(lm.fit, train)", "cat(\"Training RMSE for the full linear model:\", round(full.model.rmse, 4), \"\\n\")", " ] }, { "cell_type": "markdown", "metadata": { "id": "eight-newman", "source": [ "## Question 3 (2 Marks)", "You find the full model complicated and try to reduce the complexity by performing bidirectional stepwise regression (https://en.wikipedia.org/wiki/Stepwise\_regression) with BIC (https://en.wikipedia.org/wiki/Bayesian\_information\_criterion).", "", "Calculate the rMSE of this new model with the function that you implemented previously. Is there anything you find unusual? Explain your findings in 100 words.", " ] }, { "cell_type": "code", "execution_count": 3, "metadata": { "vscode": { "languageId": "r", "outputs": [ { "name": "stdout", "output_type": "stream", "text": [ "Training RMSE for the full linear model: 6.7017", "Training RMSE for the BIC stepwise model: 7.3303", "Number of coefficients in full model: 68", "Number of coefficients in BIC model: 15", "Variables retained by BIC:", " [1] \"income10k - 15k\" \"income120k - 150k\"", " [3] \"income150k - 200k\" \"income15k - 20k\"", " [5] \"income200k above\" \"income20k - 50k\"", " [7] \"income50k - 80k\" \"income80k - 120k\"", " [9] \"alwaysStressed\""] ] } ] }
```

"alwaysHaveFun" ", "[11] "alwaysSerious" "alwaysDepressed" ", "[13] "iFindMostThingsAmusing"

"iUsuallyHaveAGoodInfluenceOnEvents""", "Unusual finding (<=100 words):", "The unusual point is that the BIC stepwise model has a higher training RMSE (7.3303) than the full model (6.7017), even though it is the model selected by the criterion. This is not a contradiction: training RMSE only measures in-sample fit, while BIC adds a strong penalty for extra parameters. The full model uses 68 coefficients and can fit noise; BIC keeps only 15 coefficients, trading some training accuracy for simpler, more stable generalisation."] }, "source": ["# Q3: Bidirectional stepwise regression using BIC.", "n <- nrow(train)", "step.bic.fit <- step(", "lm.fit", "direction = "both", "k = log(n)", "trace = 0", "step.bic.rmse <- predict_and_rmse(step.bic.fit, train)", "cat("Training RMSE for the full linear model:", round(full.model.rmse, 4), "n")", "cat("Training RMSE for the BIC stepwise model:", round(step.bic.rmse, 4), "n")", "cat("Number of coefficients in full model:", length(coef(lm.fit)), "n")", "cat("Number of coefficients in BIC model:", length(coef(step.bic.fit)), "n")", "cat("Variables retained by BIC:\n")", "print(names(coef(step.bic.fit)) [-1])", "cat("\nUnusual finding (<=100 words):\n")", "cat(paste(" ", "The unusual point is that the BIC stepwise model has a higher training RMSE", " ", paste0("(", round(step.bic.rmse, 4), ") than the full model (", round(full.model.rmse, 4), ")", " ", "even though it is the model selected by the criterion. This is not a contradiction:", " ", "training RMSE only measures in-sample fit, while BIC adds a strong penalty for extra", " ", "parameters. The full model uses 68 coefficients and can fit noise; BIC keeps only", " ", paste(length(coef(step.bic.fit)), "coefficients, trading some training accuracy for simpler, more stable generalisation.", " ", "n")"] }, { "cell_type": "markdown", "metadata": { "id": "latter-translation" }, "source": ["## Question 4 (2 Mark)", "Although stepwise regression has reduced the model complexity significantly, the model still contains a lot of variables that we want to remove. Therefore, you are interested in lightweight linear regression models with ONLY TWO predictors. Write a script to automatically find the best lightweight model which corresponds to the model with the least **RMSE** on the training dataset. Compare the **RMSE** of the best lightweight model with the **RMSE** of the full model - `lm.fit` - that you built previously. Give an explanation for these results based on consideration of the predictors involved."] }, { "cell_type": "code", "execution_count": 4, "metadata": { "vscode": { "languageId": "r" }, "outputs": [{ "name": "stdout", "output_type": "stream", "text": ["Best two-predictor model:", "predictor_1 predictor_2 train_rmse", "income alwaysStressed 7.885411", "RMSE of best lightweight model: 7.8854 ", "RMSE of full model: 6.7017 ", "Top 10 two-predictor models:", "predictor_1 predictor_2 train_rmse", "income alwaysStressed 7.885411", "income iUsuallyHaveAGoodInfluenceOnEvents 8.099345", "income lifeIsGood 8.100236", "income alwaysHaveFun 8.116909", "income alwaysDepressed 8.137129", "income iAlwaysHaveACheerfulEffectOnOthers 8.141979", "income iAmWellSatisfiedAboutEverythingInMyLife 8.197937", "income iFindMostThingsAmusing 8.235282", "income iDoNotThinkThatTheWorldIsAGoodPlace 8.245189", "income iFeelThatLifeIsVeryRewarding 8.254198"] }] }, "source": ["# Q4: Exhaustive search over all two-predictor linear models.", "predictor.names <- setdiff(names(train), "happiness)", "predictor.pairs <- combn(predictor.names, 2, simplify = FALSE)", "pair.results <- data.frame(", "predictor_1 = character()", "predictor_2 = character()", "train_rmse = numeric()", "stringsAsFactors = FALSE", "best.rmse <- Inf", "best.pair <- NULL", "best.light.fit <- NULL", "for (pair in predictor.pairs) {", "form <- as.formula(paste("happiness ~", paste(pair, collapse = " + ")))", "fit <- lm(form, data = train)", "this.rmse <- predict_and_rmse(fit, train)", "pair.results <- rbind(", "pair.results", "data.frame(predictor_1 = pair[1], predictor_2 = pair[2], train_rmse = this.rmse)", ")", "if (this.rmse < best.rmse) {", "best.rmse <- this.rmse", "best.pair <- pair", "best.light.fit <- fit", ")", "pair.results <- pair.results[jorder(pair.results\$train_rmse),]", "cat("Best two-predictor model:\n")", "print(pair.results[1,], row.names = FALSE)", "cat("\nRMSE of best lightweight model:", round(best.rmse, 4), "n")", "cat("RMSE of full model:", round(full.model.rmse, 4), "n")", "cat("\nTop 10 two-predictor models:\n")", "print(head(pair.results, 10), row.names = FALSE)"] }, { "cell_type": "markdown", "metadata": { "id": "J4P9QL6tg7ix" }, "source": ["### ANSWER (TEXT)", "The exhaustive search selected `income` and `alwaysStressed` as the best two-predictor model. This is sensible: `income` is a strong demographic proxy in this dataset, and the fitted full model also shows many income dummy variables with very small p-values. `alwaysStressed` has a direct psychological link to happiness and remains significant in the full regression. The lightweight model has higher RMSE than the full model because it can only use these two broad signals, while the full model can combine many survey responses and factor levels. I used an exhaustive pair search rather than manual selection, so the reported model is reproducible."] }, { "cell_type": "markdown", "metadata": { "id": "preceding-chicago" }, "source": ["## Question 5 (Libraries are allowed) (40 Marks)", "As a Data Scientist, one of the key tasks is to build models *textbf{most appropriate/closest}* to the truth; thus, modelling will not be limited to the aforementioned steps in this assignment. To simulate for a realistic modelling process, this question will be in the form of a Kaggle competition (<https://www.kaggle.com/t/5a57b472c81e416cb6853ac62c422176>) among students to find out who has the best model.", "Thus, you **will be graded** by the **RMSE** performance of your model, the better your model, the higher your score. Additionally, you need to describe/document your thought process in this model building process, this is akin to showing your working properly for the mathematic sections. If you don't clearly document the reasonings behind the model you use, we will have to make some deductions on your scores.", "This is the video tutorial (https://www.youtube.com/results?search_query=how+to+join+a+kaggle+competition) on how to join any Kaggle competition.", "When you optimize your model's performance, you can use any supervised model that you know and feature selection might be a big help as well. Check the non-exhaustive set of R functions relevant to this unit (https://docs.google.com/spreadsheets/d/1Np4Zd93k7JEMyO3OZkEXq-h5BIFnGt_bs84q8u4iiv/edit?usp=sharing) for ideas for different models to try.", "textbf{Note}

Please make sure that we can install the libraries that you use in this part, the code structure can be:", "install.packages("some package", repos="http://cran.us.r-project.org")", "library("some package")", "Remember that if we cannot run your code, we will have to give you a deduction. Our suggestion is for you to use the standard R version 3.6.1", "You also need to name your final model `fin.mod` so we can run a check to find out your performance. A good test for your understanding would be to set the previous

textbf{BIC model}

to be the final model to check if your code works perfectly."] }, { "cell_type": "code", "execution_count": 5, "metadata": { "vscode": { "languageId": "r" }, "outputs": [{ "name": "stdout", "output_type": "stream", "text": ["Regression 5-fold CV RMSE:", "ridge ranger weighted_ensemble ", "8.0312 7.5014 7.4478 ", "Selected ensemble weights:", "ridge ranger ", "0.23 0.77"] }] }, "source": ["if (dir.exists("r_libs")) .libPaths(c(normalizePath("r_libs"), .libPaths()))", "install_if_missing <- function(pkg) {", "if (!requireNamespace(pkg, quietly = TRUE)) {", "try(install.packages(pkg, repos = "https://cloud.r-project.org" (https://cloud.r-project.org)), silent = TRUE)", "}", "}" }", "## Q5: Final regression model for Kaggle.", "Libraries are allowed here. The final model is selected with 5-fold CV.", "install_if_missing("ranger")", "install_if_missing("xgboost")", "train <- read.csv("regression_train.csv")", "set.seed(5197)", "make_model_matrix <- function(data, target = NULL, stored_levels = NULL) {", "x <- data", "if (is.null(target)) x[[target]] <- NULL", "if (is.null(stored_levels)) {", "stored_levels <- lapply(x, function(col) if (is.character(col)) sort(unique(col)) else

```

NULL)),""} for (nm in names(x)) {" if (is.character(x[[nm]])) x[[nm]] <- factor(x[[nm]], levels = stored_levels[[nm]])," }" list(matrix =
model.matrix(~ . - 1, data = x), levels = stored_levels),"","fit_ridge_base <- function(x, y, lambda_grid = 10 ^ seq(-3, 4, length.out = 60)) {"
x_mean <- colMeans(x)," x_sd <- apply(x, 2, sd)," x_sd[x_sd == 0] <- 1"," xs <- scale(x, center = x_mean, scale = x_sd)," y_mean <- mean(y)"
yc <- y - y_mean"," folds <- sample(rep(1:5, length.out = nrow(x)))," best_lambda <- lambda_grid[1]," best_score <- Inf"," for (lambda in
lambda_grid) {" fold_rmse <- numeric(5)," for (fold in 1:5) {" valid <- folds == fold"," xt <- xs[!valid, , drop = FALSE]," xv <- xs[valid, , drop =
FALSE]," beta <- solve(crossprod(xt) + diag(lambda, ncol(xt)), crossprod(xt, yc[!valid]))"," fold_rmse[fold] <- rmse(as.vector(y_mean + xv %*%
beta), y[valid])"," }"," if (mean(fold_rmse) < best_score) {" best_score <- mean(fold_rmse)," best_lambda <- lambda"," }"," }" beta <-
solve(crossprod(xs) + diag(best_lambda, ncol(xs)), crossprod(xs, yc))"," list(beta = beta, x_mean = x_mean, x_sd = x_sd, y_mean = y_mean,
lambda = best_lambda),""," predict_ridge_base <- function(model, x) {" xs <- scale(x, center = model$x_mean, scale = model$x_sd),"
as.vector(model$y_mean + x$model$beta),""," predict.final_regression_model <- function(object, newdata, ...) {" mm <-
make_model_matrix(newdata, stored_levels = object$levels)$matrix"," missing_cols <- setdiff(object
columns, colnames(mm))\n "," if (length(missing_cols) > 0)\n "," mm <- cbind(mm, matrix(0, nrow(mm), length(missing_cols), dimen
columns, drop = FALSE),""," pred_matrix <- matrix(NA_real_, nrow(newdata), length(object
models))\n "," colnames(pred_matrix) <- names(object$models)," pred_matrix[, "ridge"] <- predict_ridge_base(object$models$ridge,
models)) {\n",
" pred_matrix[, \"ranger\"] <-
mm)," if ("ranger" %in% names(object$models)) {"
models)) {\n",
" pred_matrix[, \"xgboost\"] <-
names(object$models) as.vector(predict(object$models$object
models$object$weights[colnames(pred_matrix)]))",""," prepared <-
make_model_matrix(train, target = "happiness")," x <- prepared$matrix\n "," y <- train$happiness"," reg_folds <- sample(rep(1:5, length.out
= nrow(train)))"," oof <- matrix(NA_real_, nrow(train), 1)," colnames(oof) <- "ridge"," if (requireNamespace("ranger", quietly = TRUE)) {" oof <-
cbind(oof, ranger = NA_real_)"," }"," for (fold in 1:5) {" valid <- reg_folds == fold"," ridge.fold <- fit_ridge_base(x[!valid, , drop = FALSE],
y[!valid])"," oof[valid, "ridge"] <- predict_ridge_base(ridge.fold, x[valid, , drop = FALSE])"," if ("ranger" %in% colnames(oof)) {" rf.fold <-
ranger::ranger(" happiness ~ .," data = train[!valid, ]," num.trees = 500," mtry = 18," min.node.size = 5," seed = 5197 + fold," )" oof[valid,
predictions\n",
" }\n",
" }\n",
" }\n",
" cv.rmse <- apply(oof, 2, rmse, actual = y)\n",
" if (ncol(oof) == 2) {\n",
" weight.grid <- seq(0, 1, by = 0.01)\n",
" ensemble.rmse <- sapply(weight.grid, function(w)
{\n",
" rmse(w * oof[, \"ridge\"] + (1 - w) * oof[,
\"ranger\"], y)\n",
" }\n",
" best.w <-
weight.grid[which.min(ensemble.rmse)]\n",
" weights <- c(ridge = best.w, ranger = 1 -
best.w)\n",
" cv.rmse <- c(cv.rmse, weighted_ensemble =
min(ensemble.rmse))\n",
" } else {\n",
" weights <- c(ridge = 1)\n",
" }\n",
" }\n",
" cat(\"Regression 5-fold CV RMSE:\n\n")\n",
" print(round(cv.rmse, 4))\n",
" cat(\"Selected ensemble weights:\n\n")\n",
" print(round(weights, 3))\n",
" }\n",
" models <- list(ridge = fit_ridge_base(x, y))\n",
" if ("ranger" %in% names(weights)) {\n",
" models
" ranger <-
ranger::ranger(" happiness ~ .," data = train," num.trees = 1000," mtry = 18," min.node.size = 5," seed = 5197," )"," fin.mod <-
list(models = models, weights = weights / sum(weights), levels = prepared

```

```

levels, columns = colnames(x))\n",
"class(fin.mod) <-
\"final_regression_model\"\n"
}
},
{
"cell_type": "code",
"execution_count": 6,
"metadata": {
"id": "j0SYx-zYg7ix",
"vscode": {
"languageId": "r"
}
},
"outputs": [],
"source": [
"# Load in the test data.\n",
"test <-
read.csv(\"regression_test.csv\")\n",
"# If you are using any packages that perform the prediction
differently, please change this line of code accordingly.\n",
"pred.label <- predict(fin.mod, test)\n",
"# put these predicted labels in a csv file that you can use to
commit to the Kaggle Leaderboard\n",
"write.csv(\n",
"  data.frame(\"RowIndex\" = seq(1,
length(pred.label)), \"Prediction\" = pred.label), \n",
"  \"RegressionPredictLabel.csv\", \n",
"  row.names = F\n",
")\n",
],
},
{
"cell_type": "code",
"execution_count": null,
"metadata": {
"id": "gross-disaster",
"outputId":
"ac8b1b5a-ad80-42f7-824a-61432e4d64cf",
"vscode": {
"languageId": "r"
}
},
"outputs": [],
"source": [
"### PLEASE DO NOT ALTER THIS CODE BLOCK. YOU ARE REQUIRED TO
HAVE THIS CODE BLOCK IN YOUR JUPYTER NOTEBOOK SUBMISSION\n",
"### Please skip (don't run) this if you are a
student\n",
"### For teaching team use only\n",
"\n",
"tryCatch(\n",
"  {\n",
"    source(\"../supplimentary.R\")\n",
"  },\n",
"  error = function(e){\n",
"    source(\"supplimentary.R\")\n",
"  }\n",
")\n",
"\n",
"truths <- tryCatch(\n",
"  {\n",
"    read.csv(\"../regression_test_label.csv\")\n",
"  },\n",
"  error = function(e){\n",
"    read.csv(\"regression_test_label.csv\")\n",
"  }\n",
")\n",
"\n",
"RMSE.fin <- rmse(pred.label,
truths

```

```

x)","cat(paste("RMSE is", RMSE.fin))" ] }, { "cell_type": "markdown",
"metadata": { "id": "f_u6Dyj1xehk" }, "source": [ "# Part 2 Classification (50 Marks)" ] }, { "cell_type": "markdown", "metadata": {}, "source": [ "A few
thousand people were questioned in a life and wellbeing survey to build a model to predict happiness of an individual, but this time we want to
predict a categorical score for their mental health, rather than a continuous score. You need to build 5-class classification models to optimally
predict the variable in the survey dataset called 'alwaysAnxious' based on any, or all, of the other survey question responses. ", "", "You have been
provided with two datasets, classification_train.csv and classification_test.csv. Using these datasets, you hope to build a model that can
predict 'alwaysAnxious' using the other variables. classification_train.csv comes with the ground-truth target label (i.e. 'alwaysAnxious'
happiness classes) whereas classification_test.csv comes with independent variables (input information) only.", "", "On the order of around 70
survey questions have been converted into predictor variables that can be used to predict 'alwaysAnxious'. We do not list all the predictor names
here, but their names given in the data header can clearly be linked to the survey questions. E.g. the predictor variable
'iDontFeelParticularlyPleasedWithTheWayIam' corresponds to the survey question 'I don't feel particularly pleased with the way I am.'", "", "This
question will also be in the form of a Kaggle competition (https://www.kaggle.com/t/fb7422ceba914c50ab67a78a59b6498f) among students to find
out who has the best model." ] }, { "cell_type": "markdown", "metadata": {}, "source": [ "### Classification final-model notes", "", "After checking
Kaggle public feedback, the best classification submissions among the tested candidates reached a public macro F1 of 0.56078. I selected two
complementary submissions for the final leaderboard: C22_xgb70_gbr30_c09_threshold.csv and C35_gbr_more_trees_c09_threshold.csv. Both
use an ordinal-threshold strategy: the five anxiety classes are treated as ordered numeric targets, a gradient-boosted regression model predicts a
continuous score, and tuned thresholds convert that score back into the five allowed classes. C22 blends XGBoost ordinal predictions with GBM
ordinal predictions, while C35 uses a more-trees GBM ordinal model. The saved ClassificationPredictLabel.csv uses the C22 version, and the
C35 version is kept as the second selected Kaggle submission." ] }, { "cell_type": "code", "execution_count": 7, "metadata": { "id": "8x-H3d66g7ix",
"vscode": { "languageId": "r" } }, "outputs": [ { "name": "stdout", "output_type": "stream", "text": [ "Class distribution in training data:\n (data:\n", "", "-2
-1 0 1 2 ","119 87 182 139 67 ","", "Single-model stratified 5-fold macro F1: ","", "ranger svm xgboost multinom "," 0.3890 0.3724 0.2037 0.3681
", "", "Best weighted probability ensemble macro F1: 0.4217 ", "Selected model weights: ","", "ranger svm xgboost multinom "," 0.775 0.016 0.141 0.067
", "", "Best class probability multipliers: "," -2 -1 0 1 2 ","1.5 1.4 1.3 1.4 1.3 ", "Final tuned ensemble macro F1: 0.4317 ", "Out-of-fold confusion matrix for

```

```

tuned ensemble:", "Predicted", "Actual -2 -1 0 1 2", "-2 32 20 50 15 2", "-1 25 32 28 2 0", "0 28 19 87 48 0", "1 10 3 37 81 8", "2 1 0 15 25 26" ] } ],
"source": [ "if (dir.exists("r_libs")) .libPaths(c(normalizePath("r_libs"), .libPaths()))", "", "install_if_missing <- function(pkg) {", "if",
(!requireNamespace(pkg, quietly = TRUE)) {", "try(install.packages(pkg, repos = "https://cloud.r-project.org\\ (https://cloud.r-project.org\\)", silent =
TRUE)", "}", "}", "# Load in the train and test classification
data.", "install_if_missing("ranger")", "install_if_missing("e1071")", "install_if_missing("xgboost")", "train <- read.csv("classification_train.csv")", "test <-
read.csv("classification_test.csv")", "set.seed(5197)", "", "for (nm in names(train)) {", "if (is.character(train[[nm]])) train[[nm]] <-
factor(train[[nm]])", "}", "trainalwaysAnxious <- ~ factor(trainalwaysAnxious, levels = sort(unique(train
alwaysAnxious)))\\n", "class.levels <- levels(trainalwaysAnxious)", "factor.levels <- lapply(train, function(col) if (is.factor(col))
levels(col) else NULL)", "", "prepare_classification_newdata <- function(newdata, levels_list) {", "for (nm in names(newdata)) {", "if (nm %in%
names(levels_list) && !is.null(levels_list[[nm]])) {", "newdata[[nm]] <- factor(newdata[[nm]], levels = levels_list[[nm]])", "}", "}",
newdata", "}", "macro_f1 <- function(actual, predicted) {", "actual <- factor(actual, levels = class.levels)", "predicted <- factor(predicted, levels =
class.levels)", "mean(sapply(class.levels, function(label) {", "tp <- sum(actual == label & predicted == label)", "fp <- sum(actual != label & predicted
== label)", "fn <- sum(actual == label & predicted != label)", "if ((2 * tp + fp + fn) == 0) return(0)", "2 * tp / (2 * tp + fp + fn)",
alwaysAnxious <- NULL\\n",
"model.matrix(~ . - 1, data = x)\\n",
"}\\n",
"}\\n",
"align_probabilities <- function(prob_matrix) {\\n",
"prob_matrix <- as.matrix(prob_matrix)\\n",
"aligned <- matrix(0, nrow(prob_matrix),
length(class.levels), dimnames = list(NULL, class.levels))\\n",
"common <- intersect(colnames(prob_matrix),
class.levels)\\n",
"aligned[, common] <- prob_matrix[, common, drop =
FALSE]\\n",
"aligned\\n",
"}\\n",
"}\\n",
"stratified_folds <- function(y, k = 5) {\\n",
"folds <- integer(length(y))\\n",
"for (label in levels(y)) {\\n",
"idx <- which(y == label)\\n",
"folds[idx] <- sample(rep(1:k, length.out =
length(idx)))\\n",
"}\\n",
"folds\\n",
"}\\n",
"}\\n",
"predict_final_classification_model <- function(object,
newdata, ...) {\\n",
"newdata <- prepare_classification_newdata(newdata,
object
)}", "}", "make_xgb_matrix <- function(data, target = FALSE) {", "x <- data", "if (target) x
models)) {\\n",
"prob.list",
ranger <- align_probabilities(predict(objectmodelsranger, data =
predictions)\\n",
"}\\n",
"if (!\"svm\" %in% names(objectmodels)) {", "svm.pred <- predict(objectmodelssvm, newdata = newdata, probability = TRUE)",
svm <-
align_probabilities(attr(svm.pred,
\"probabilities\"))\\n",
"}\\n",
"if (!\"xgboost\" %in%
names(objectmodels)) {", "xnew <- make_xgb_matrix(newdata)", "missing.cols <- setdiff(object
xgb.columns, colnames(xnew))\\n", "if (length(missing.cols) > 0)\\n", "xnew <- cbind(xnew, matrix(0, nrow(xnew), length(missing.
xgb.columns, drop = FALSE)", "xp <- matrix(predict(objectmodelsxgboost, xgboost:xgb.DMatrix(xnew)), ncol = length(class.levels), byrow =
xgboost <- xpl\\n",
"}\\n",
"if (!\"multinom\" %in%
names(objectmodels)) {", "prob.list
TRUE)", "colnames(xp) <- class.levels", "prob.list
names(objectmodels)) {", "prob.list
multinom, newdata = newdata, type =
\"probs\")\\n",
"}\\n",
"}\\n",
"avg <- matrix(0, nrow(newdata), length(class.levels),
dimnames = list(NULL, class.levels))\\n",
"for (model.name in names(prob.list)) {\\n",
"avg <- avg + object
weights[model.name] *
class_multipliers)\\n",
"as.integer(as.character(class.levels[max.col(adjusted,
ties.method = \"first\")]))\\n",
"}\\n",
"}\\n",
"models.to.try <- c(\"ranger\", \"svm\",
\"xgboost\", \"multinom\")\\n",
"cls.folds <- stratified_folds(train
alwaysAnxious, 5)", "oof <- array(0,
dim = c(nrow(train), length(class.levels), length(models.to.try))", "dimnames = list(NULL, class.levels, models.to.try))", "for (fold in 1:5) {", "valid <-
cls.folds == fold", "tr <- train[!valid, ]", "va <- train[valid, ]", "valid.x <- va[, setdiff(names(va), \"alwaysAnxious\")]", "if (requireNamespace(\"ranger\",
quietly = TRUE)) {", "rf.fold <- ranger::ranger(alwaysAnxious ~ ., data = tr, num.trees = 700, mtry = 15, min.node.size = 3, probability = TRUE,
predictions)\\n",
"}\\n",
"}\\n",
"if (requireNamespace(\"e1071\", quietly = TRUE))
{\\n",
"class_counts <- table(tr
seed = 5197 + fold)", "oof[valid, \"ranger\"] <- align_probabilities(predict(rf.fold, data = valid.x)
alwaysAnxious)", "class_weights <- as.numeric(sum(class_counts) / (length(class_counts) * class_counts))", "names(class_weights) <-
names(class_counts)", "svm.fold <- e1071::svm(alwaysAnxious ~ ., data = tr, kernel = \"radial\", cost = 1, gamma = 0.03, class.weights =
class_weights, scale = TRUE, probability = TRUE)", "svm.pred <- predict(svm.fold, newdata = valid.x, probability = TRUE)", "oof[valid, \"svm\"] <-
align_probabilities(attr(svm.pred, \"probabilities\"))", "}", "if (requireNamespace(\"xgboost\", quietly = TRUE)) {", "xtr <- make_xgb_matrix(tr, target =
TRUE)", "xva <- make_xgb_matrix(valid.x)", "missing.cols <- setdiff(colnames(xtr), colnames(xva))", "if (length(missing.cols) > 0) {", "xva <-
cbind(xva, matrix(0, nrow(xva), length(missing.cols), dimnames = list(NULL, missing.cols)))", "}", "xva <- xva[, colnames(xtr), drop = FALSE]",

```

```
xgb.fold <- xgboost::xgb.train(" ", params = list(objective = "multi:softprob", num_class = length(class.levels), "eval_metric" = "mlogloss",
max_depth = 2, eta = 0.025, "subsample" = 0.85, colsample_bytree = 0.85, lambda = 8, alpha = 0.2), "data = xgboost::xgb.DMatrix(xtr, label =
```

```
alwaysAnxious) - 1)\n",
  "  rounds = 700,\n",
  "  verbose = 0\n",
  ")\n",
  "  xp <- matrix(predict(xgb.fold,
xgboost::xgb.DMatrix(xva)), ncol = length(class.levels), byrow =
TRUE)\n",
  "  colnames(xp) <- class.levels\n",
  "  oof[valid, , \"xgboost\"] <- xp\n",
  ")\n",
  ")\n",
  "  if (requireNamespace(\"nnet\", quietly = TRUE))
{\n",
  "    mn.fold <- nnet::multinom(alwaysAnxious ~ ., data =
tr, trace = FALSE, MaxNWts = 10000, maxit = 1000)\n",
  "    oof[valid, , \"multinom\"] <-
align_probabilities(predict(mn.fold, newdata = valid.x, type =
\"probs\"))\n",
  "  }\n",
  ")\n",
  ")\n",
  "  \"available.models <- models.to.try[sapply(models.to.try,
function(m) sum(oof[, , m]) > 0)]\n",
  "  \"single.f1 <- sapply(available.models, function(m)
{\n",
  "    macro_f1(train
```

```
alwaysAnxious, class.levels[max.col(oof[, , m], ties.method =
\"first\")])\" , \" )\" , \" \" , \"set.seed(5197)\", \"best.score <- -Inf\", \"best.weights <- rep(1 / length(available.models),
length(available.models))\" , \"names(best.weights) <- available.models\", \"for (i in 1:15000) {\", \" raw.weights <- rexp(length(available.models))\",
weights <- raw.weights / sum(raw.weights)\", \" names(weights) <- available.models\", \" avg <- matrix(0, nrow(train), length(class.levels), dimnames =
list(NULL, class.levels))\", \" for (model.name in available.models) avg <- avg + weights[model.name] * oof[, , model.name]\", \" score <- macro_f1(train
```

```
alwaysAnxious,
class.levels[max.col(avg, ties.method = \"first\")])\n",
  "  if (score > best.score) {\n",
  "    best.score <- score\n",
  "    best.weights <- weights\n",
  "  }\n",
  ")\n",
  ")\n",
  "  \"avg <- matrix(0, nrow(train), length(class.levels),
dimnames = list(NULL, class.levels))\n",
  "  \"for (model.name in available.models) avg <- avg +
best.weights[model.name] * oof[, , model.name]\n",
  "  \"\n",
  "  \"best.mult <- rep(1, length(class.levels))\n",
  "  \"names(best.mult) <- class.levels\n",
  "  \"best.mult.score <- best.score\n",
  "  \"mult.grids <- list(seq(0.7, 1.8, by = 0.1), seq(0.7, 1.8,
by = 0.1),\n",
  "    seq(0.7, 1.5, by = 0.1), seq(0.7, 1.6, by =
0.1),\n",
  "    seq(0.7, 2.3, by = 0.1))\n",
  "  \"for (a in mult.grids[[1]]) for (b in mult.grids[[2]]) for (c
in mult.grids[[3]])\n",
  "    for (d in mult.grids[[4]]) for (e in mult.grids[[5]])
{\n",
  "    mult <- c(a, b, c, d, e)\n",
  "    pred <- class.levels[max.col(t(t(avg) * mult),
ties.method = \"first\")]\n",
  "    score <- macro_f1(train
```

```
alwaysAnxious, pred)\", \" if (score > best.mult.score) {\", \" best.mult.score <- score\", \" best.mult <-
mult\", \" names(best.mult) <- class.levels\", \" }\", \" }\", \" ensemble.pred <- class.levels[max.col(t(t(avg) * best.mult), ties.method = \"first\")]\", \" cat(\"Class
distribution in training data:\\n\\ (data:\\n\\)\", \" print(table(train
alwaysAnxious))\\n \" , \" cat(\"
```

```
nSingle – modelstratified5 – foldmacroF1 :
```

```
n\\")\\n \" , \" print(round(single.f1, 4))\\n \" , \" cat(\"
```

```
nBestweightedprobabilityensemblemacroF1 : \", round(best.score, 4), \"
```

```
n\\")\\n \" , \" cat(\"Selectedmodelweights :
```

```
n\\")\\n \" , \" print(round(best.weights, 3))\\n \" , \" cat(\"Bestclassprobabilitymultipliers :
```

```
n\\")\\n \" , \" print(round(best.mult, 2))\\n \" , \" cat(\"FinaltunedensemblemacroF1 : \", round(best.mult.score, 4), \"
```

```
n\\")\\n \" , \" cat(\"Out – of – foldconfusionmatrixfortunedensemble :
```

```
n\\")\\n \" , \" print(table(Actual = train
```

```
alwaysAnxious, Predicted = factor(ensemble.pred, levels = class.levels)))\", \" , \" final.models <- list()\", \" if (\"ranger\" %in% available.models) {\", \"
```

```
ranger <- ranger::ranger(alwaysAnxious ~ .,
data = train, num.trees = 1000,\n",
  "  mtry = 15,
min.node.size = 3, probability = TRUE, seed = 5197)\n",
  ")\n",
  "  \"if (\"svm\" %in% available.models) {\n",
  "    class_counts <- table(train
```

```
alwaysAnxious)\", \" class_weights <- as.numeric(sum(class_counts) /
(length(class_counts) * class_counts))\", \" names(class_weights) <- names(class_counts)\", \" final.models
```

```

svm <-
e1071::svm(alwaysAnxious ~ ., data = train, kernel =
\"radial\",
"
cost = 1, gamma = 0.03,
class.weights = class_weights,\"n\",
"
scale = TRUE, probability =
TRUE)\n",
")\n",
"rgb.columns <- NULL\n",
"if (\"xgboost\" %in% available.models) {\n",
" xfull <- make_xgb_matrix(train, target = TRUE)\n",
" xgb.columns <- colnames(xfull)\n",
" final.models

```

```

xgboost <- xgboost::xgb.train(," params = list(objective = "multi:softprob", num_class =
length(class.levels),," eval_metric = "mlogloss", max_depth = 2, eta = 0.025,," subsample = 0.85, colsample_bytree = 0.85, lambda = 8, alpha =

```

```

alwaysAnxious) -
1),\n",
" nrounds = 700,\n",
" verbose = 0\n",
" )\n",
")\n",
"if (\"multinom\" %in% available.models) {\n",
" final.models

```

```

0.2),," data = xgboost::xgb.DMatrix(xfull, label = as.integer(train
nnet::multinms(alwaysAnxious ~ ., data = train, trace = FALSE, MaxNWts = 10000, maxit = 1000),"",," fin.mod <- list(models = final.models,
weights = best.weights, class_multipliers = best.mult,," levels = factor.levels, xgb.columns = xgb.columns)","class(fin.mod) <-
"final_classification_model","",,"# If you are using any packages that perform the prediction differently, please change this line of code
accordingly.",,"pred.label <- predict(fin.mod, test)","# put these predicted labels in a csv file that you can use to commit to the Kaggle
Leaderboard","write.csv("," data.frame("RowIndex" = seq(1, length(pred.label)), "Prediction" = pred.label),," "ClassificationPredictLabel.csv",,"
row.names = F",," ) }, { "cell_type": "code", "execution_count": null, "metadata": { "vscode": { "languageId": "r" } }, "outputs": [], "source": [ "##
PLEASE DO NOT ALTER THIS CODE BLOCK, YOU ARE REQUIRED TO HAVE THIS CODE BLOCK IN YOUR JUPYTER NOTEBOOK
SUBMISSION", "## Please skip (don't run) this if you are a student", "## For teaching team use only", "", "truths <- tryCatch(", " ",
read.csv("../classification_test_label.csv"),",",," error = function(e){", " read.csv("classification_test_label.csv"),",",,"),",," f1_score <-
F1_Score(truths$x, pred.label)","cat(paste("f1_score is", f1_score))" } } ], "metadata": { "colab": { "collapsed_sections": [], "name":
"FIT5197_Assignment_Marking_Guidelines.ipynb", "provenance": [] }, "kernelspec": { "display_name": "R", "language": "R", "name": "ir" },
"language_info": { "codemirror_mode": "r", "file_extension": ".r", "mimetype": "text/x-r-source", "name": "R", "pygments_lexer": "r", "version": "3.6.1" }
}, "nbformat": 4, "nbformat_minor": 4 }

```